



Islamic University of Technology

Department of Computer Science and Engineering

SWE 4801: Software Maintenance

Summer, 2024–2025

Lab 02: Code Property Graph Generation

Saadman Sakib

ID: 210042165

Program: B.Sc. Engg. in SWE

Department of Computer Science and Engineering

Islamic University of Technology

Date: August 21, 2026

Contents

1	Introduction	2
2	Background	2
2.1	Code Property Graph (CPG)	2
2.2	Joern	2
2.3	Neo4j	2
3	Target Application	3
3.1	Design Rationale	3
3.2	Project Structure	3
3.3	Features That Enrich the CPG	3
4	Tool Installation	4
4.1	Joern CLI (Pre-built Release)	4
4.2	Neo4j Desktop	4
5	CPG Generation and Export	4
5.1	Importing Source Code	4
5.2	Interactive Verification	4
5.3	Export to Neo4j CSV	5
6	Importing into Neo4j	5
6.1	Copying Files to the Import Folder	5
6.2	Fixing File URI References	5
6.3	Enabling File Import in Neo4j	5
6.4	Running the Import	6
7	Graph Exploration in Neo4j Browser	6
7.1	Verification	6
7.2	Connected Graph View	7
7.3	Method Nodes	7
7.4	AST of a Specific Function	8
7.5	Control Flow Graph	8
7.6	Data Dependency Edges	8
8	Problems Encountered	9
8.1	Git Clone Failure (HTTP/2 Stream Reset)	9
8.2	SSL Connection Error	9
8.3	sbt Stage Build Failure	9
8.4	Joern Workspace in System32	9
8.5	Incomplete File Copy to Neo4j Import Folder	10
8.6	Single-Slash URI in Joern Cypher Files	10
8.7	Neo4j File Import Disabled by Default	10
9	Conclusion	10

1 Introduction

Code Property Graphs (CPGs) provide a unified, queryable representation of source code by merging multiple program analysis structures into a single graph model [1]. This lab demonstrates the end-to-end process of generating a CPG from a real Node.js backend application, exporting it, and importing it into a graph database for visual exploration.

The toolchain used consists of **Joern** [2] as the CPG generator and **Neo4j** as the graph database and visualisation platform. The source code analysed is a purpose-built CRUD REST API [3] designed to produce a structurally varied graph with multiple layers of control flow, data dependency, and cross-model relationships.

2 Background

2.1 Code Property Graph (CPG)

A CPG is a composite graph structure that unifies three classical program representations [1]:

- **Abstract Syntax Tree (AST):** Captures the hierarchical syntactic structure of source code - how statements and expressions are nested inside one another.
- **Control Flow Graph (CFG):** Captures the order in which statements execute, including branches from `if/else` conditions and loops.
- **Program Dependence Graph (PDG):** Captures data and control dependencies between statements, i.e., how values defined in one location are consumed in another.

By overlaying these three graphs on a shared node set, a CPG enables expressive queries spanning syntax, control flow, and data flow simultaneously — making it well-suited for vulnerability detection, software maintenance analysis, and program comprehension [1].

2.2 Joern

Joern is an open-source platform for code analysis built around the CPG model [2]. It provides language frontends that parse source code (JavaScript, C/C++, Java, Python, and others) into a CPG binary, an interactive Scala-based REPL for querying the graph, and the `joern-export` utility for converting the binary CPG into formats consumable by external graph databases.

2.3 Neo4j

Neo4j is a native graph database that stores data as nodes and typed relationships rather than relational tables [2]. In this lab it serves two purposes: persisting the exported CPG as a queryable database, and providing visual graph exploration through the Neo4j Browser interface using the Cypher query language.

3 Target Application

3.1 Design Rationale

A minimal CRUD application with linear function calls produces a CPG that is technically correct but visually sparse. The target application was therefore designed to generate a richer graph with meaningful branching, data flow, and cross-model dependencies. The full source is available at [3].

3.2 Project Structure

The application is a Node.js/Express/MongoDB REST API:

Listing 1: Project directory structure

```

1 crud-app/
2   config/db.js           # MongoDB connection
3   models/
4     user.model.js        # User schema (name, email, age)
5     post.model.js        # Post schema (title, body, author ref,
6                           tags)
7   controllers/
8     user.controller.js    # CRUD + search + cascade delete
9     post.controller.js    # CRUD + publish toggle + tag filter
10  routes/
11    user.routes.js
12    post.routes.js
13  utils/validation.js     # validateUser, validatePost,
                           buildUserQuery
14  server.js               # Entry point

```

3.3 Features That Enrich the CPG

Table 1: Code patterns and their CPG impact

Feature	CPG Impact
Multi-branch <code>if/else</code> in <code>validateUser</code>	Deep CFG branching with multiple execution paths
Conditional query building in <code>buildUserQuery</code>	PDG data-flow edges from <code>req.query</code> to DB call
Nested <code>async</code> calls in <code>createUser</code>	Multi-level AST call trees
Cascade delete across two models	Cross-model data dependency edges
Read-then-mutate in <code>togglePublish</code>	Explicit read/write PDG dependency
<code>ObjectId</code> reference between Post and User	Inter-model relationship nodes

4 Tool Installation

4.1 Joern CLI (Pre-built Release)

Installation was performed on Windows 11 (HP, AMD Ryzen 5, 20 GB RAM). Building from source via `sbt stage` was attempted first but failed due to network-level blocking of Maven and Gradle repositories (see Section 8). The pre-built release was used instead [2].

1. Downloaded `joern-cli.zip` from <https://github.com/joernio/joern/releases/latest>
2. Extracted to `C:\joern-cli`
3. Added `C:\joern-cli` to the Windows system `PATH` via Environment Variables
4. Verified with `joern --version`; Joern displays an ASCII art banner and launches the REPL

4.2 Neo4j Desktop

Neo4j Desktop was installed from <https://neo4j.com/download/>. A local DBMS named `crud-cpg` was created through the Desktop GUI, started, and accessed via the built-in Neo4j Browser at `http://localhost:7474`.

5 CPG Generation and Export

5.1 Importing Source Code

The Joern REPL was launched via `joern` from CMD. The CRUD application was imported with:

Listing 2: Importing source code into the Joern REPL

```
1 joern> importCode("C:/crud-app")
```

Joern's JavaScript frontend parsed all `.js` files and saved the CPG binary to its workspace. Because Joern was launched from `C:\Windows\System32`, the workspace was written there and located via:

Listing 3: Locating the generated CPG binary

```
1 C:\> dir /s /b cpg.bin 2>nul
2 C:\Windows\System32\workspace\crud-app\cpg.bin
```

5.2 Interactive Verification

The CPG layers were verified in the REPL before exporting:

Listing 4: Verifying CPG contents in the REPL

```
1 joern> cpg.file.name.1
2 joern> cpg.method.name.1
3 joern> cpg.controlStructure.controlStructureType("IF").code.1
```

5.3 Export to Neo4j CSV

Listing 5: Exporting CPG to Neo4j CSV format

```
1 joern-export.bat --repr all --format neo4jcsv ^  
2 --out C:\cpg-output\export ^  
3 C:\Windows\System32\workspace\crud-app\cpg.bin
```

Result:

Listing 6: Export summary

```
1 exported 4631 nodes, 32949 edges into C:\cpg-output\export
```

The output directory contains paired `_data.csv` and `_cypher.csv` files for each node and edge label.

6 Importing into Neo4j

6.1 Copying Files to the Import Folder

All files from the export directory were copied to the Neo4j DBMS import folder:

Listing 7: Copying all export files to Neo4j import directory

```
1 copy "C:\cpg-output\export\*.*" ^  
2 "C:\Users\HP\Neo4jDesktop2\Data\dbmss\dbms-b15e0e4e-...\import\"
```

6.2 Fixing File URI References

The `_cypher.csv` files use single-slash URIs (`file:/nodes_X_data.csv`), which Neo4j does not recognise. A PowerShell script patched all files to use `file:///`:

Listing 8: Patching file URIs in all cypher CSV files

```
1 Get-ChildItem "C:\cpg-output\export\*_cypher.csv" | ForEach-Object {  
2     $content = Get-Content $_.FullName -Raw  
3     $fixed = $content -replace "file:/nodes_", "file:///nodes_"  
4     $fixed = $fixed -replace "file:/edges_", "file:///edges_"  
5     Set-Content $_.FullName $fixed  
6     Write-Host "Fixed: $($_.Name)"  
7 }
```

6.3 Enabling File Import in Neo4j

The following settings were enabled in `neo4j.conf` via the Neo4j Desktop settings editor, followed by a database restart:

Listing 9: neo4j.conf: enabling CSV file loading

```
1 server.directories.import=import  
2 dbms.security.allow_csv_import_from_file_urls=true
```

6.4 Running the Import

Nodes and edges were imported via `cypher-shell` using PowerShell loops:

Listing 10: Importing node CSV files via `cypher-shell`

```
1 $shell = "C:\Users\HP\Neo4jDesktop2\Data\dbmss\dbms-b15e0e4e-...\bin
   \cypher-shell.bat"
2 $password = "password123"
3
4 Get-ChildItem "C:\cpg-output\export\nodes*_cypher.csv" | ForEach-
   Object {
5     Write-Host "Importing nodes: $($_.Name)"
6     & $shell -u neo4j -p $password --file $_.FullName
7 }
```

Listing 11: Importing edge CSV files via `cypher-shell`

```
1 Get-ChildItem "C:\cpg-output\export\edges*_cypher.csv" | ForEach-
   Object {
2     Write-Host "Importing edges: $($_.Name)"
3     & $shell -u neo4j -p $password --file $_.FullName
4 }
```

Sample output confirming successful import:

Listing 12: Sample import output

```
1 Importing edges: edges_ARGUMENT_cypher.csv
2 Created 2327 relationships
3 Importing edges: edges_AST_cypher.csv
4 Created 4401 relationships
```

7 Graph Exploration in Neo4j Browser

The Neo4j Browser was accessed at <http://localhost:7474>. All queries below were executed in the Cypher query box.

7.1 Verification

Listing 13: Verifying total imported node count

```
1 MATCH (n) RETURN count(n)
```

Returned **4631**, confirming a complete import.

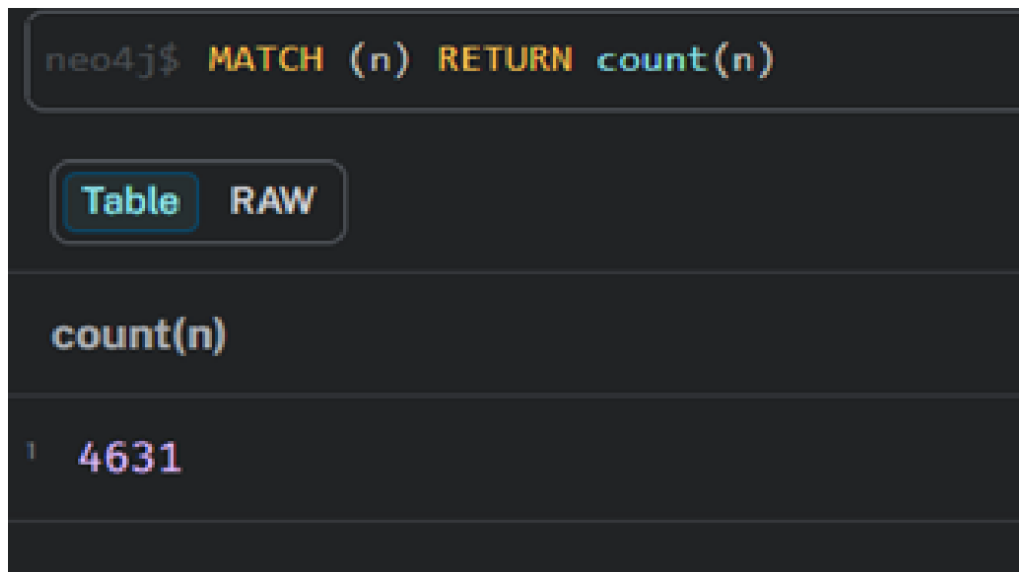


Figure 1: Verification of total node count returned by Neo4j Browser

7.2 Connected Graph View

Listing 14: Visualising node-edge-node triples

```
1 MATCH (a)-[r]->(b)
2 RETURN a, r, b LIMIT 100
```

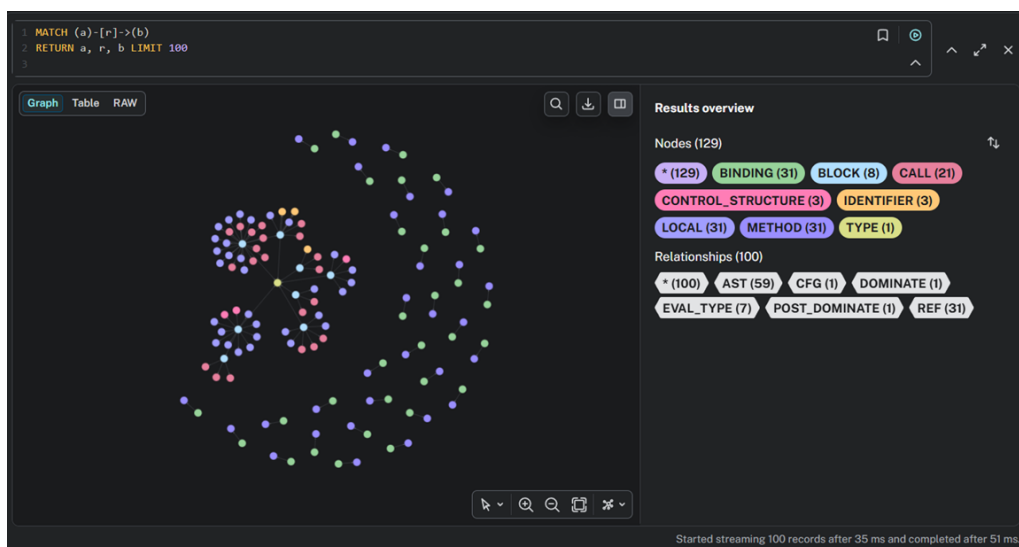


Figure 2: Neo4j Browser graph view showing nodes connected by labelled edges (LIMIT 100)

7.3 Method Nodes

Listing 15: Listing all detected methods

```
1 MATCH (n:METHOD) RETURN n.name, n.filename
```

7.4 AST of a Specific Function

Listing 16: AST subtree of createUser (depth 1–5)

```
1 MATCH (m:METHOD {name: "createUser"})-[:AST*1..5]->(child)
2 RETURN m, child
```

7.5 Control Flow Graph

Listing 17: CFG edges showing execution paths

```
1 MATCH (a)-[r:CFG]->(b)
2 RETURN a, r, b LIMIT 80
```

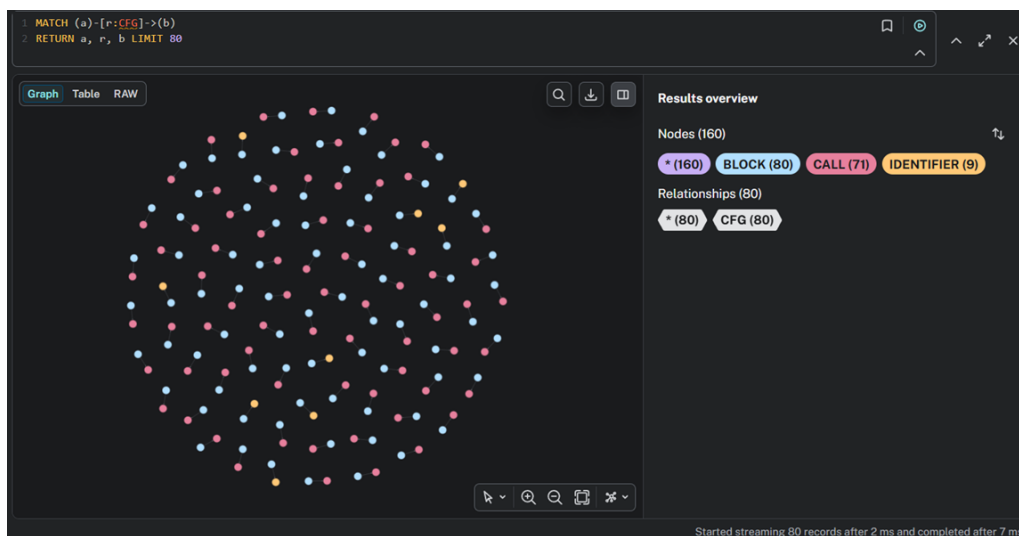


Figure 3: CFG edges visualised in Neo4j Browser, showing branching execution paths from if-chains in validateUser and buildUserQuery

7.6 Data Dependency Edges

Listing 18: PDG REACHING_DEF edges (data flow)

```
1 MATCH (a)-[r:REACHING_DEF]->(b)
2 RETURN a, r, b LIMIT 80
```

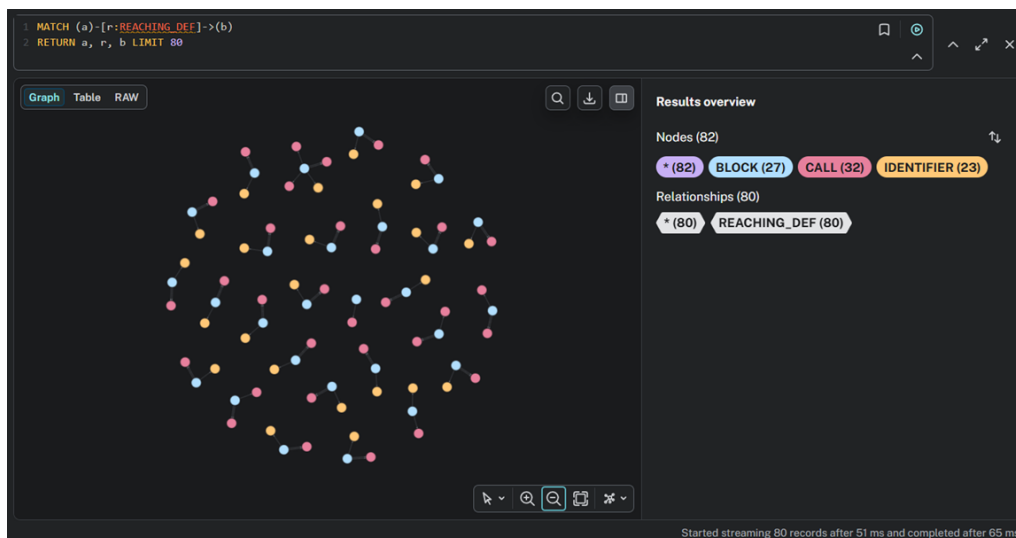


Figure 4: PDG REACHING_DEF edges in Neo4j Browser, showing data flow from variable definitions to their uses across function boundaries

8 Problems Encountered

8.1 Git Clone Failure (HTTP/2 Stream Reset)

Cloning the Joern repository failed with an HTTP/2 stream cancellation error. Increasing the Git post buffer and downgrading to HTTP/1.1 resolved the protocol mismatch:

Listing 19: Fixing Git clone via buffer and protocol settings

```
1 git config --global http.postBuffer 524288000
2 git config --global http.version HTTP/1.1
```

8.2 SSL Connection Error

A subsequent clone attempt produced `OpenSSL SSL_read: SSL_ERROR_SYSCALL, errno 0`, indicating ISP or firewall-level SSL interception. **Resolution:** Switched to downloading `joern-cli.zip` directly via browser, bypassing Git entirely.

8.3 sbt Stage Build Failure

Building from source with `sbt stage` failed after 21 minutes with `UnknownHostException` for `repo.gradle.org` and Atlassian Maven repositories — both unreachable due to DNS blocking. **Resolution:** Used the pre-built Joern CLI release.

8.4 Joern Workspace in System32

Joern was launched from the default CMD directory (`C:\Windows\System32`), so the workspace and `cpg.bin` were written there. A filesystem search (`dir /s /b cpg.bin`) was needed to locate the file.

8.5 Incomplete File Copy to Neo4j Import Folder

Only `_cypher.csv` files were initially copied, omitting the `_data.csv` files they reference. This caused 22N43: `unable to load external resource` errors on every import. **Resolution:** All files (`*.*`) were copied.

8.6 Single-Slash URI in Joern Cypher Files

Joern generates `file:/nodes_X_data.csv` URIs (one slash), whereas Neo4j requires `file:///` (three slashes). Even with all files present, imports failed until all cypher files were patched with the PowerShell find-and-replace script described in Section 6.2.

8.7 Neo4j File Import Disabled by Default

Neo4j Desktop blocks filesystem CSV loading by default. Enabling `dbms.security.allow_csv_import_from_file_paths.enabled` in `neo4j.conf` and restarting the DBMS was required before any import succeeded.

9 Conclusion

This lab demonstrated the complete pipeline for generating and visualising a Code Property Graph from JavaScript source code. Using Joern's pre-built CLI, a Node.js/Express/MongoDB CRUD application was parsed into a CPG containing **4631 nodes** and **32949 edges**, covering AST, CFG, and PDG layers. The graph was exported to Neo4j CSV format, imported into a local Neo4j Desktop instance, and explored via Cypher queries that revealed method structures, control flow branches, and data dependency edges.

The lab also highlighted practical challenges in deploying open-source analysis tooling in network-restricted environments, and the importance of understanding how tools manage workspace state, file paths, and security defaults.

References

- [1] Ghaffarian, S.M. et al., *Program Structure-aware Language Models: Targeted Software Testing beyond Textual Semantics*, arXiv:2604.17715v1 [cs.SE], 2026. Available: <https://arxiv.org/abs/2604.17715>
- [2] CoderPad, *Code Property Graph, Oriented Databases, & Source Code Analysis*, CoderPad Blog, 2024. Available: <https://coderpad.io/blog/development/code-property-graph-oriented-databases-source-code-analysis/>
- [3] Saadman Sakib, *crud-app: Node.js/Express/MongoDB CRUD API for CPG Analysis*, GitHub, 2025. Available: <https://github.com/saadmansakib47/crud-app>
- [4] Anthropic, *Claude AI — Installation Assistance and Guidance*, 2025. Available: <https://claude.ai>